

Good. We keep the product fixed:

**AlphaTrace = Trader PNL leaderboard + smart-money movement intelligence.**

**“Which Solana wallets are actually profitable, and what are those wallets doing now?”**

# 1. Architectural north star

**Birdeye raw data → wallet normalization → wallet scoring → wallet ranking → wallet profiling → token-level smart-money aggregation → UI/API presentation.**

So the system has two major intelligence layers:

1. **Wallet Intelligence Layer**

This answers:

**“Is this wallet actually good?”**

2. **Token Signal Layer**

This answers:

**“What tokens are good wallets touching right now?”**

Everything else exists to support those two layers.

---

# 2. System boundary

AlphaTrace has a tight boundary.

It does **not** execute trades.

It does **not** custody user funds.

It does **not** connect wallets for signing.

It does **not** claim to predict price.

It is an **onchain analytics and intelligence product**.

The system consumes Birdeye data, computes derived metrics, caches results, and exposes trader-facing views.

So the architecture is read-heavy, API-heavy, computation-light, and cache-sensitive.

That matters because our core engineering problems are:

- API credit management
  - latency
  - data freshness
  - wallet ranking correctness
  - derived-score explainability
  - avoiding fake “smart money” signals
  - presenting complex wallet data simply
- 

### 3. High-level architecture

The clean architecture:

Frontend



App API Layer



Domain Services



Birdeye Client + Cache



Birdeye API

But more specifically:

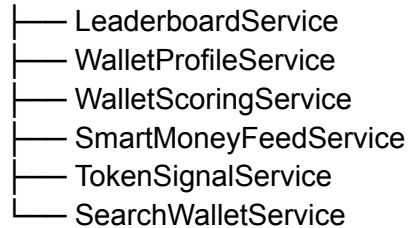
Next.js / React UI



API Routes / Backend Server

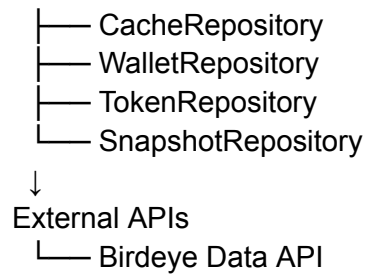


Application Services



Infrastructure Layer





The frontend should never know Birdeye endpoint details.

The UI should ask our backend:

- “Give me leaderboard”
- “Give me wallet profile”
- “Give me smart-money feed”
- “Analyze this wallet”

The backend decides which Birdeye endpoints to call, how to cache, how to enrich, and how to score.

That separation is important

## 4. The core data flow

### Flow 1: Build leaderboard

This is the homepage data flow.

User opens homepage



Frontend calls GET /api/leaderboard?window=7d



LeaderboardService checks cache



If stale:

- call /trader/gainers-losers
- extract candidate wallets
- enrich each wallet with PNL summary
- fetch wallet age
- compute Alpha Score
- persist snapshot



Return ranked wallets

The leaderboard is not just the Birdeye response. It is your enriched ranking.

Core Birdeye endpoints:

- `/trader/gainers-losers`
- `/wallet/v2/pnl/summary`
- `/wallet/v2/pnl/details`
- `/wallet/v2/tx/first-funded`

The architectural point:

**Birdeye gives candidate wallets. AlphaTrace gives interpreted rankings.**

---

## Flow 2: Build wallet profile

This is what happens when a user clicks a wallet.

User clicks wallet



Frontend calls GET `/api/wallet/:address/profile`



WalletProfileService checks cache



If stale:

- fetch PNL summary
- fetch PNL details
- fetch current holdings
- fetch net worth history
- fetch first funded date
- fetch token metadata for traded/held tokens
- compute wallet archetype
- compute Alpha Score explanation



Return profile object

Core endpoints:

- `/wallet/v2/pnl/summary`

- `/wallet/v2/pnl/details`
- `/wallet/v2/net-worth`
- `/v1/wallet/token_list`
- `/wallet/v2/tx/first-funded`
- `/defi/v3/token/meta-data/multiple`

The wallet profile is a composed read model.

## Flow 3: Build smart-money feed

This is the product's most important architecture.

Scheduled job / user request



Take top N wallets from leaderboard



For each wallet:

inspect holdings

inspect balance changes / recent activity

inspect PNL credibility



Group activity by token



For each token:

count smart wallets involved

calculate combined Alpha Score of wallets

fetch token market data

fetch token security status

fetch token metadata

compute Smart Money Signal Score



Return ranked token signals

Core endpoints:

- `/v1/wallet/token_list`
- `/wallet/v2/balance-change`
- `/wallet/v2/pnl`
- `/wallet/v2/pnl/details`
- `/defi/v3/token/market-data`

- `/defi/token_security`
- `/defi/v3/token/meta-data/single` or `/multiple`

The architectural point:

**The smart-money feed is an aggregation over wallets, not a token scanner.**

That distinction matters.

A token scanner starts from tokens.

AlphaTrace starts from **credible wallets**, then asks what tokens appear in their behavior.

---

## 5. Backend services

Keep the backend around a few serious modules.

### 5.1 BirdeyeClient

This is your only direct integration with Birdeye.

Responsibilities:

- Add API key
- Add chain parameter
- Normalize errors
- Handle rate limits
- Retry safe failures
- Track API credit usage
- Apply endpoint-specific typing
- Log requests

It should expose methods like:

```
getTraderGainersLosers(window)
getWalletPnlSummary(wallet, window)
getWalletPnlDetails(wallet, window)
getWalletNetWorth(wallet, window)
getWalletTokenList(wallet)
getWalletFirstFunded(wallet)
getTokenMetadata(tokens[])
```

```
getTokenMarketData(token)
getTokenSecurity(token)
```

Do not scatter raw HTTP calls across the app.

## 5.2 LeaderboardService

This builds the ranked wallet list.

Responsibilities:

- Fetch candidate wallets
- Remove duplicates
- Filter suspicious wallets
- Enrich wallets
- Compute Alpha Score
- Sort and paginate
- Persist leaderboard snapshots

Important design choice:

**Do not compute the leaderboard from scratch on every page load.**

That will burn API credits and make the app slow.

Use a snapshot model:

```
leaderboard_snapshot
  id
  window
  generated_at
  wallet_count
  source
```

```
leaderboard_entries
  snapshot_id
  wallet_address
  rank
  pnl_usd
  roi_percent
  win_rate
  volume_usd
  alpha_score
  wallet_age_days
```

archetype

Then the homepage reads from your own database/cache.

---

## 5.3 WalletProfileService

This builds one wallet's profile.

Responsibilities:

- Fetch wallet PNL summary
- Fetch token-level PNL details
- Fetch net worth history
- Fetch holdings
- Fetch token metadata
- Compute profile metrics
- Compute archetype
- Return explanation

This service should produce a stable response like:

WalletProfile

wallet

summary

alphaScore

alphaScoreBreakdown

archetype

netWorthSeries

topWinningTokens

topLosingTokens

currentHoldings

recentActivity

The UI should not have to calculate any of this.

---

## 5.4 WalletScoringService

This is one of the most important modules.



It converts raw wallet stats into your own intelligence layer.

Input:

pnlSummary  
pnlDetails  
walletAge  
recentActivity  
holdings

Output:

alphaScore  
scoreBreakdown  
archetype  
riskFlags  
confidence

The score must be explainable.

Example:

Alpha Score: 82

Breakdown:

- Realized PNL: strong
- ROI: strong
- Consistency: medium
- Wallet age: strong
- Recent activity: active
- Risk penalty: low

Do not just return 82.

Return **why 82**.

That gives the product credibility.

---

## 5.5 SmartMoneyFeedService

This service answers:

**“What are top wallets touching?”**

Responsibilities:

- Load top wallets from latest leaderboard snapshot
- Pull current holdings and/or recent balance changes
- Detect token overlap
- Aggregate per token
- Call TokenSignalService for scoring
- Return ranked feed

Output should look like:

SmartMoneySignal  
token  
tokenSymbol  
tokenLogo  
signalScore  
smartWalletCount  
combinedWalletPnl  
averageAlphaScore  
netFlowDirection  
marketData  
securitySummary  
involvedWallets[]  
explanation

Example explanation:

**“5 high-alpha wallets have exposure to this token. Average wallet Alpha Score is 78. Combined historical PNL is \$430k. Token security check passed. Liquidity is acceptable.”**

That sentence is the product.

---

## 5.6 TokenSignalService

This computes token-level signal strength.

It should not discover every token on Solana. Scope stays tight.

It only scores tokens that appear in smart-wallet activity.

Inputs:

- involved smart wallets
- wallet alpha scores
- token market data
- token security status
- token metadata

Outputs:

- smart money signal score
- label
- warning flags
- explanation

Signal labels:

| Score  | Label                       |
|--------|-----------------------------|
| 80–100 | Strong Smart Money Signal   |
| 60–79  | Emerging Smart Money Signal |
| 40–59  | Weak / Noisy                |
| 0–39   | Ignore                      |

This lets the UI explain what the user is seeing.

---

## 5.7 SearchWalletService

This powers the viral “search your wallet” feature.

Responsibilities:

- Validate wallet address
- Fetch wallet PNL
- Compare to latest leaderboard distribution
- Assign approximate percentile
- Generate trader archetype
- Return share-card payload

Important: the user's wallet may not be in the leaderboard. So do not promise exact rank unless it is in your tracked set.

Use language like:

- “Top 8% among analyzed wallets”
- “Compared against current AlphaTrace sample”
- “Ranked #42 in current tracked set”

Architecturally, this avoids lying.

---

## 6. Data model

You need three kinds of data:

1. **Raw-ish snapshots**
2. **Derived wallet metrics**
3. **Derived token signals**

### 6.1 Wallet table

wallets

address

first\_seen\_at

first\_funded\_at

source

last\_refreshed\_at

Purpose:

Track wallets you have encountered.

Sources may be:

- trader\_gainers\_losers
  - top\_traders
  - user\_search
  - smart\_money\_feed
-

## 6.2 Wallet metrics snapshot

wallet\_metric\_snapshots

id  
wallet\_address  
window  
pnl\_usd  
roi\_percent  
win\_rate  
trade\_count  
volume\_usd  
wallet\_age\_days  
alpha\_score  
alpha\_class  
confidence  
generated\_at

Purpose:

Wallet stats change over time. You do not want only the latest value. Snapshots let you show progress and avoid recomputing constantly.

---

## 6.3 Wallet token PNL

wallet\_token\_pnl  
wallet\_address  
token\_address  
window  
realized\_pnl\_usd  
unrealized\_pnl\_usd  
roi\_percent  
buy\_count  
sell\_count  
volume\_usd  
last\_activity\_at  
generated\_at

Purpose:

This powers “best trades,” “worst trades,” and token-level wallet explanations.

---

## 6.4 Wallet holdings

wallet\_holdings  
  wallet\_address  
  token\_address  
  balance  
  value\_usd  
  portfolio\_weight  
  updated\_at

Purpose:

This powers current portfolio and smart-money exposure.

---

## 6.5 Token metadata

tokens  
  address  
  symbol  
  name  
  logo\_uri  
  decimals  
  chain  
  updated\_at

Purpose:

Avoid repeatedly fetching token labels.

---

## 6.6 Token signal snapshot

token\_signal\_snapshots  
  id  
  token\_address  
  signal\_score  
  signal\_label  
  smart\_wallet\_count

combined\_wallet\_pnl  
average\_alpha\_score  
liquidity\_usd  
volume\_24h  
security\_status  
generated\_at

Purpose:

This powers the Smart Money Feed.

---

## 6.7 Token signal participants

token\_signal\_participants  
signal\_snapshot\_id  
wallet\_address  
wallet\_alpha\_score  
wallet\_pnl\_usd  
holding\_value\_usd  
portfolio\_weight  
reason

Purpose:

This explains why the token appeared.

Without participants, your token feed becomes a black box.

With participants, every signal is auditable.

---

## 7. Caching architecture

This project lives or dies on caching.

Birdeye API calls are valuable. Do not waste them.

Use three cache layers.

## Layer 1: Request cache

Short-lived memory cache.

TTL:

- token metadata: 24 hours
- token market data: 1–5 minutes
- wallet PNL summary: 10–30 minutes
- wallet holdings: 2–5 minutes
- leaderboard: 10–30 minutes
- smart-money feed: 5–15 minutes

## Layer 2: Database snapshots

This is the main cache.

The app can always show the last generated snapshot, even if refresh fails.

That improves reliability.

## Layer 3: Background refresh

A cron job refreshes:

- leaderboard every 15–30 minutes
- top wallet profiles every 30–60 minutes
- smart-money feed every 10–15 minutes
- token metadata lazily

For the bounty, we can run this as a server-side scheduled task or manual refresh endpoint. But architecturally, treat it as a separate worker.

---

# 8. API design

Keep your API small and clean.

## 8.1 Leaderboard

GET /api/leaderboard?window=24h|7d|30d&limit=50&page=1



Returns:

```
{
  "window": "7d",
  "generatedAt": "...",
  "entries": [
    {
      "rank": 1,
      "wallet": "...",
      "pnlUsd": 128000,
      "roiPercent": 240,
      "winRate": 0.71,
      "alphaScore": 91,
      "alphaClass": "Elite Alpha",
      "walletAgeDays": 430,
      "archetype": "Alpha Hunter"
    }
  ]
}
```

---

## 8.2 Wallet profile

GET /api/wallets/:address/profile?window=7d

Returns:

```
{
  "wallet": "...",
  "summary": {},
  "alphaScore": {},
  "archetype": {},
  "netWorthSeries": [],
  "topWinningTokens": [],
  "topLosingTokens": [],
  "currentHoldings": [],
  "riskFlags": []
}
```

---

## 8.3 Smart-money feed

GET /api/smart-money/feed?window=24h&limit=30

Returns:

```
{
  "generatedAt": "...",
  "signals": [
    {
      "token": "...",
      "symbol": "XYZ",
      "signalScore": 84,
      "label": "Strong Smart Money Signal",
      "smartWalletCount": 5,
      "combinedWalletPnl": 430000,
      "averageAlphaScore": 78,
      "securityStatus": "passed",
      "explanation": "...",
      "participants": []
    }
  ]
}
```

---

## 8.4 Search wallet

GET /api/wallets/:address/analyze?window=30d

Returns:

```
{
  "wallet": "...",
  "pnlUsd": 12800,
  "roiPercent": 64,
  "alphaScore": 72,
  "percentile": 0.18,
  "archetype": "Rotator",
  "shareCard": {
    "title": "I scored 72 on AlphaTrace",
    "subtitle": "Top 18% of analyzed Solana wallets"
  }
}
```

```
}
```

---

## 8.5 Health

GET /api/health

Returns:

```
{
  "ok": true,
  "birdeyeReachable": true,
  "lastLeaderboardRefresh": "...",
  "lastSmartMoneyRefresh": "...",
}
```

This is good presentation discipline.

---

## 9. Frontend architecture

The frontend should map directly to the product concepts.

### Page 1: Leaderboard

Purpose:

**Rank wallets.**

Components:

- WindowSelector
- LeaderboardTable
- WalletRankRow
- AlphaScoreBadge
- ArchetypeBadge
- SearchWalletInput

---

## Page 2: Wallet profile

Purpose:

**Explain one wallet.**

Components:

- `WalletHeader`
- `PerformanceSummaryCards`
- `AlphaScoreBreakdown`
- `NetWorthChart`
- `TokenPnlTable`
- `HoldingsTable`
- `WalletArchetypePanel`

The profile page should answer:

- Is this wallet profitable?
  - Was it lucky or consistent?
  - What tokens made it money?
  - What does it hold now?
  - Is it still active?
- 

## Page 3: Smart-money feed

Purpose:

**Convert wallet intelligence into token intelligence.**

Components:

- `SmartMoneySignalList`
- `TokenSignalCard`
- `SmartWalletParticipants`
- `SignalScoreBadge`
- `SecurityMiniBadge`

- `TokenMarketStats`

Each card should be explanation-first.

Not just:

**“Token XYZ score 84.”**

But:

**“Token XYZ appears because 5 high-alpha wallets currently hold it or recently accumulated it.”**

---

## Page 4: Wallet search / trader card

Purpose:

**Virality and user acquisition.**

Components:

- `WalletSearchInput`
- `TraderCard`
- `ShareButton`
- `ComparableStats`

This page should be visually clean and shareable.

---

# 10. Scoring architecture

The scoring should be deterministic, not AI-generated.

That means:

Same input → same score.

## 10.1 Alpha Score

Alpha Score =

realized\_pnl\_score \* 0.25  
+ roi\_score \* 0.20  
+ win\_rate\_score \* 0.15  
+ consistency\_score \* 0.15  
+ wallet\_age\_score \* 0.10  
+ recency\_score \* 0.10  
- risk\_penalty \* 0.05

Each sub-score should be normalized to 0–100.

## 10.2 Confidence score

Separately compute confidence.

Why?

A wallet with only 2 trades can have 100% win rate. That does not make it elite.

Confidence should depend on:

- number of trades
- wallet age
- number of tokens traded
- volume
- data completeness

Output:

alphaScore: 88  
confidence: "medium"

This prevents shallow ranking.

## 10.3 Archetype classifier

Use rule-based classification.

Examples:

### Alpha Hunter

- high ROI
- strong realized PNL

- many small/midcap wins

### **Consistent Compounder**

- moderate ROI
- high win rate
- stable net worth curve

### **One-Hit Wonder**

- high PNL
- low consistency
- one token dominates profits

### **Exit Liquidity**

- negative PNL
- poor win rate
- high losses

### **Inactive Winner**

- good historical PNL
- low recent activity

This makes the product memorable.

---

## **11. Smart-money signal architecture**

This deserves special attention.

The feed should not say:

**“This token is good.”**

It should say:

**“This token has smart-wallet attention.”**

That is legally and intellectually cleaner.

## **Token signal computation**

For each token found among top wallets:

Signal Score =  
smart\_wallet\_count\_score \* 0.25  
+ average\_alpha\_score \* 0.20  
+ combined\_pnl\_score \* 0.20  
+ portfolio\_weight\_score \* 0.10  
+ market\_liquidity\_score \* 0.10  
+ security\_score \* 0.10  
+ recency\_score \* 0.05

## Token signal explanation

Every signal should include:

- how many smart wallets are involved
- average Alpha Score of those wallets
- combined historical PNL of those wallets
- whether the token passes basic security checks
- whether liquidity is acceptable
- whether the signal is fresh or stale

This is the difference between a toy dashboard and a real analytics product.

---

## 12. Background job architecture

Even for an MVP, think in jobs.

### Job 1: Refresh leaderboard

Runs every 15–30 minutes.

fetch candidate wallets  
enrich wallets  
score wallets  
save leaderboard snapshot

### Job 2: Refresh top wallet profiles



Runs every 30–60 minutes.

- take top 50 wallets
- refresh PNL details
- refresh holdings
- refresh net worth
- save profile cache

## **Job 3: Refresh smart-money feed**

Runs every 10–15 minutes.

- take top wallets
- extract held/recent tokens
- group by token
- score token signals
- save feed snapshot

## **Job 4: Refresh token metadata**

Lazy/background.

- for unknown tokens:
  - fetch metadata
  - save token info

For a hackathon/competition implementation, jobs can be triggered manually with admin endpoints:

- POST /api/admin/refresh-leaderboard
- POST /api/admin/refresh-smart-money-feed

But architecturally, they are still jobs.

---

# **13. Error handling architecture**

You need graceful degradation.

Possible failures:

- Birdeye rate limit
- endpoint timeout
- wallet has no PNL data
- token metadata missing
- malformed wallet address
- incomplete net worth history
- security endpoint unavailable

Do not let one missing endpoint kill the whole profile.

Use partial responses:

```
{
  "data": {...},
  "warnings": [
    "Net worth history unavailable",
    "Token security check unavailable for 2 tokens"
  ],
  "freshness": {
    "pnl": "fresh",
    "holdings": "cached",
    "metadata": "cached"
  }
}
```

This makes the system robust.

---

## 14. Deployment architecture

For speed:

Vercel frontend + API routes

Supabase Postgres

Upstash Redis optional

Cron via Vercel Cron / Supabase scheduled function

Or:

Next.js app  
Postgres  
Prisma  
Server-side scheduled refresh

Minimal but solid:

Next.js  
TypeScript  
Postgres/Supabase  
Prisma  
TanStack Query  
Recharts  
Birdeye API

Do not use too many services. The product is already complex enough.

---

## 15. What your repo structure should look like

```
alphatrace/  
  app/  
    page.tsx  
  leaderboard/  
    wallet/[address]/  
    smart-money/  
  api/  
    leaderboard/  
    wallets/[address]/profile/  
    wallets/[address]/analyze/  
    smart-money/feed/  
    health/  
  components/  
    leaderboard/  
    wallet/  
    smart-money/  
    shared/  
  lib/  
    birdeye/
```

client.ts  
endpoints.ts  
types.ts  
scoring/  
  walletAlphaScore.ts  
  tokenSignalScore.ts  
  archetypes.ts  
services/  
  leaderboardService.ts  
  walletProfileService.ts  
  smartMoneyFeedService.ts  
  tokenSignalService.ts  
repositories/  
  walletRepository.ts  
  snapshotRepository.ts  
  tokenRepository.ts  
cache/  
  cacheKeys.ts  
  ttl.ts  
prisma/  
  schema.prisma  
docs/  
  architecture.md  
  scoring-model.md  
  birdeye-endpoints.md

The `docs/` folder is important for Presentation score. The bounty explicitly values presentation and GitHub quality.

---

## 16. The mental model

Birdeye gives you low-level data:

- wallet PNL
- token holdings
- token metadata
- token security
- top traders
- wallet net worth

Your system creates high-level read models:

- leaderboard row
- wallet profile
- trader card
- token signal card

A read model is a pre-shaped object designed for the UI.

That is the architecture.

Not:

**“Call endpoint and render JSON.”**

But:

**“Build domain-specific views from multiple endpoint compositions.”**

---

## 17. Final architecture summary

AlphaTrace has four architectural layers:

### 1. Data acquisition layer

BirdeyeClient fetches wallet, token, PNL, metadata, and security data.

### 2. Normalization layer

Raw API responses become clean internal objects:

- Wallet
- Token
- WalletPNL
- Holding
- TokenSignalParticipant

### 3. Intelligence layer

Your system computes:

- Alpha Score
- confidence
- wallet archetype
- smart-money signal score
- token signal explanation

## **4. Presentation layer**

The frontend renders:

- leaderboard
- wallet profile
- smart-money feed
- wallet search/trader card